CS 650 – Full Stack Application Develop

1-2 Short Paper: Core Data Entities and Storage Strategy

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

July 9, 2026

Data Modeling Proposal for the Setlist Application

The first step in designing any application is understanding what users want to accomplish and identifying the information the system must store to support those tasks. A well-designed data model helps ensure that the application is efficient, scalable, and easy to maintain as new features are added. For the Setlist application, the primary goal is to provide professional session musicians with a simple way to organize their personal song library and create setlists for upcoming performances. The application allows users to add songs, organize them into multiple setlists, update information when needed, and delete songs or setlists when they are no longer required. To support these workflows, the system must store song information, setlist information, and the relationship between songs and setlists.

User Goals, Actions, and Required Information

The application's primary users are individual musicians who need an organized way to prepare for performances. Their main goal is to maintain a library of songs and build customized setlists for different events. Based on the product description, users should be able to create, read, update, and delete songs from their library. They should also be able to create new setlists, assign songs to one or more setlists, remove songs from individual setlists, and delete completed setlists after an event. To support these activities, the application must store information about each song, including the title, artist, musical key, tempo, and duration. The system must also store information about each setlist, including the event name, performance date, venue, and optional notes. Finally, the application needs a way to connect songs and setlists because one song may appear in multiple setlists, and each setlist can contain many songs.

Core Data Entities

The proposed SQLite database contains three primary entities: Song, Setlist, and SetlistSong.

Entity 1: Song

Attribute	**SQLite Data Type**	**Constraint**
SongID	INTEGER	Primary Key, AUTOINCREMENT
Title	TEXT	NOT NULL
Artist	TEXT	NOT NULL
MusicalKey	TEXT	NOT NULL
Tempo	INTEGER	NOT NULL
Duration	INTEGER	NOT NULL

The Song entity stores information about every song in the musician's personal library. Each song receives a unique SongID that serves as the primary key.

Entity 2: Setlist

Attribute	SQLite Data Type	Constraint
SetlistID	INTEGER	Primary Key, AUTOINCREMENT
Name	TEXT	NOT NULL
EventDate	TEXT	NOT NULL
Venue	TEXT	NOT NULL
Notes	TEXT	NULL Allowed

The Setlist entity stores information about each performance or event. The Notes field is optional because musicians may not always need to include additional comments.

Entity 3: SetlistSong (Junction Table)

Attribute	SQLite Data Type	Constraint
SetlistID	INTEGER	Foreign Key
SongID	INTEGER	Foreign Key
SongOrder	INTEGER	NOT NULL
Primary Key	(SetlistID, SongID)	Composite Primary Key

The SetlistSong table resolves the many-to-many relationship between songs and setlists. It also stores the order in which songs should be performed during an event.

Relationships Between the Entities

The relationship between Song and Setlist is many-to-many. A musician may use the same song in several performances, while each performance typically includes multiple songs. Because relational databases cannot directly represent many-to-many relationships, the SetlistSong table acts as a junction table connecting the two entities. These relationships are essential for the application's workflow. Without the junction table, users would have to create duplicate song records every time they wanted to reuse a song in another performance. Instead, the application stores each song only once and references it whenever it is added to a setlist. This design reduces duplicate data, improves consistency, and makes updates much easier. For example, if the musician changes the tempo or key of a song, the updated information automatically appears everywhere the song is used.

Storage Approach

SQLite is an appropriate storage solution for this application because the system is designed for individual use rather than supporting thousands of simultaneous users. SQLite stores the entire database in a single file, making it lightweight, portable, and easy to deploy. Since the application primarily manages structured information with clearly defined relationships, a relational database provides an excellent fit. Foreign key constraints help maintain data integrity by ensuring that every song assigned to a setlist actually exists within the song library (Silberschatz et al., 2020).

Another advantage of SQLite is its strong support for CRUD operations. Creating new songs or setlists requires simple INSERT statements. Users can retrieve information efficiently with SELECT queries, update existing records using UPDATE statements, and remove outdated information with DELETE statements. Because the database is normalized, these operations remain straightforward while reducing unnecessary duplication (Elmasri & Navathe, 2017).

Tradeoff of the Storage Approach

Although SQLite is an excellent choice for this application, it does have limitations. SQLite performs very well for single-user desktop or lightweight web applications, but it is not designed for large numbers of concurrent users making frequent updates. If the application eventually expanded into a cloud-based platform supporting thousands of musicians at the same time, a database management system such as PostgreSQL or MySQL would provide better scalability and concurrency. For the current application, however, SQLite's advantages outweigh its limitations. CRUD operations remain efficient because the amount of stored data is relatively small, and the normalized database structure simplifies data management while maintaining consistency. This allows the application to provide reliable performance without the complexity of managing a larger database server.

To conclude, designing an effective data model at the beginning of a project establishes a strong foundation for future development. The proposed Song, Setlist, and SetlistSong entities support all of the workflows described in the product requirements while maintaining data integrity and minimizing redundancy. SQLite provides a practical storage solution because it is lightweight, easy to maintain, and well-suited for applications intended for individual users. Together, these

design decisions create a flexible database structure that supports efficient CRUD operations while allowing the application to evolve as additional features are introduced.

References

Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of database systems* (7th ed.). Pearson.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill Education.